

Scheduling techniques — a comparative study

E P K Tsang

Techniques in both operations research (OR) and artificial intelligence (AI) have been demonstrated to be useful in certain scheduling problems. Some of the better known scheduling techniques developed in OR are linear programming (LP), branch-and-bound (BB) and tabu search. In AI, better known scheduling techniques are hill climbing (HC), simulated annealing (SA), constraint satisfaction (CS), genetic algorithms (GAs), connectionist methods and expert systems (ES).

Given that different techniques have different natures, it is reasonable to believe that they have different domains in which they are effective. A major obstacle which prevents problem solvers (people who have real problems to solve) from applying these techniques to scheduling is the difficulty in knowing when to apply which technique. It is argued that expertise in 'which technique to apply when' is at least as important as expertise in the individual techniques, if not more so.

This paper summarises and contrasts some of the better known OR and AI techniques for scheduling, and, based on their nature, suggests guidelines for problem solvers in choosing between them

1. Introduction

A scheduling problem mainly involves resource allocation. This normally takes the form of assigning values to a set of variables, satisfying a set of constraints and possibly optimising a certain function. The following is an example of a scheduling problem which will be used later in this paper.

Problem: to produce each of the four products P1 to P4, jobs must be done by some or all of the four workshops, W1 and W4. Each workshop may only process one job at a time, and are only available in certain time periods. Each product must be processed by the workshops in the order specified for it and each workshop may take different amounts of time to process different jobs. A job must be done in a continuous period. (Variations of the problem may allow jobs to be done intermittently.) Furthermore, each product can only start after a certain time and must be produced before a deadline. Table 1 summarises the production requirements.

Figure 2 shows an example schedule which satisfies all the constraints in the example problem. Apart from satisfying the constraints, some applications require optimisation. For example, it may be preferable to finish all the jobs as early as possible, or, when it is not possible to satisfy all the constraints, one may be allowed to pay penalties for violating certain constraints (e.g. paying overtime to the workshops in the above example). Thenthe

task is to produce a schedule which incurs the minimum amount of penalty.

Table 1 Production requirements.

Products	Earliest starting time	Jobs: workshop ordering and processing tiems (in hours)	Deadlines
P1	7:00	W1(2), W2(1), W3(3), W4(1)	20:00
P2	9:00	W2(3), W3(1), W4(2)	18:00
P3	9:00	W1(1), W2(4), W4(3)	21:00
P4	8:00	W3(2), W1(2), W4(1), W2(2)	19:00

Techniques in both operations research (OR) and artificial intelligence (AI) have been demonstrated to be useful in certain scheduling problems. Classic scheduling techniques developed in OR are linear (LP,[1]) and branch-and-bound (BB) [5,6]. Better known scheduling techniques in AI are hill climbing (HC), simulated annealing [10], genetic algorithms (GAs) [15, 16] and expert systems [17, 18]. Tabu search and some of these AI techniques are referred to as advanced information processing (AIP) techniques. Different techniques take different approaches to tackle a problem; this will be elaborated later in this paper. It is reasonable to believe that different techniques have different domains in which they are especially effective.

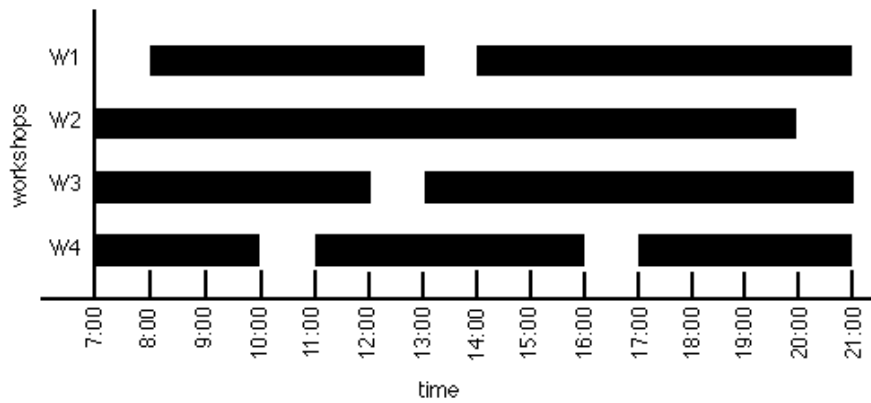


Fig 1 Availability of workshops.

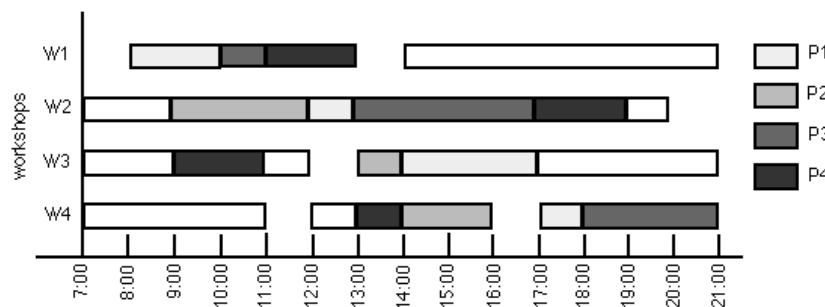


Fig 2 An example schedule which satisfies all the constraints.

From the point of a problem solver (someone who has a real problem to solve), knowing which technique to use for a particular problem is extremely difficult. In fact, most problem solvers will simply use the technique with which they are most familiar for all the problems that they encounter.

One of the main arguments in this paper is that expertise in 'which technique to apply when' is at least as important as expertise in the individual techniques, if not more so. This paper first summarises and then contrasts the ways that some of the best known OR and AI techniques are applied to scheduling. Then, based on the nature of these techniques, guidelines will be suggested to help problem solvers choose between them when facing a problem.

2. Where to start when tackling a scheduling problem

One of the major goals of this paper is to help problem solvers to retrieve appropriate methods for tackling scheduling problems. This section looks at the issue of where to start. Unfortunately, work on similar goals is rare; in addition, it is next to impossible to make generalisations and provide rigid rules and steps for indexing methods. Hopefully this paper provides a practical starting point.

It is proposed that the answers to the following questions are useful in helping to choose between the various techniques for a given scheduling problem.

- Does the problem require any solution which satisfies all the constraints (e.g. to allocate time slots in the workshops for processing the products, satisfying all the constraints, in the above example), or does it require some sort of optimisation (e.g. to find a schedule which finishes all the jobs as early as possible, or, if not all the constraints can be satisfied, to find a schedule which violates the least number of constraints)? Often, satisfiability problems can be formulated as optimisation problems, where the function to be minimised is the number of constraints being violated.
- If optimisation is required, is the function to be optimisation linear?
- If optimisation is required, are near-optimal solutions acceptable?
- How much time does the program have for finding solutions? Is the time available in seconds, hours or days? Are sub-optimal solutions more useful than optimal solutions which take a long time to produce?
- What sort of domain specific knowledge is available?

In this paper, it was suggested that the answers to these questions will help to guide the problem solver to choose between the various techniques. This will be dealt with in more detail later in this paper.

3. Summary of AIP scheduling techniques

This section will look at a number of AIP techniques for scheduling. For reference, LP and BB will also be included in the summary. The example scheduling problem in section 1 will be used to contrast these techniques. An attempt will also be made to identify situations in which each of these techniques are particularly useful (focusing on the answers to the above questions), highlighting the factors which affect the effectiveness of each of them. Constraint programming systems which embed AIP techniques will also be briefly described.

3.1 Linear programming

Linear programming (LP) is normally applicable to problems which can be described by a conjunctive set of linear equations and/or inequalities. The task is to optimise a given linear function. Therefore, to apply LP to a scheduling problem, one must first establish a set of inequalities and define a function to be optimised. If the given problem requires the optimisation of a certain linear function, then that function can be used by LP for optimisation. If the task is simply to satisfy all the constraints, then a dummy optimization function can be used. After a problem is formulated, one can apply standard LP techniques to find the optimal schedule.

One possible way to tackle the example problem in section 1 by LP is to create variables which represent the starting time of each job. In the example problem above, there are 14 jobs to be done in total. So, 14 variables are to be created. Let T_{xy} represent the job of processing product P_x by workshop y and D_{xy} can only start by 7:00, inequality (1) below applies:

$$T_{11} \geq 7 \quad \text{..... (1)}$$

Since workshop T_1 takes 2 hours to process P_1 , and W_2 can only process P_1 after W_2 , inequality (2) below applies:

$$T_{11} + 2 \leq T_{12} \quad \text{.....(2)}$$

To ensure that only one job can be processed by a workshop at a time, a number of inequalities need to be set up. To ensure that W_y does not process P_x and P_x simultaneously, either inequality (3) or (4) below must hold:

$$T_{xy} + D_{xy} \leq T_{x'y} \quad \text{..... (3)}$$

$$18 \quad T_{x'y} \leq D_{x'y} + T_{xy} \quad \text{..... (4)}$$

If inequality (3) holds, W_y processes P_x before $P_{x'}$. If inequality (4) holds, then W_y processes $P_{x'}$ before P_x . Unfortunately, this creates disjunctive inequalities (only one of (3) or (4) need to hold). Since similar inequalities must be set up for each pair of products processed by every workshop, there could be a large number of them. Disjunctive inequalities are normally handled by enumeration — every combination of inequalities will form a conjunctive set of inequalities and be handled individually.

To complete the formulation of the problem, an optimisation function is required. In this problem, if the requirement is simply to satisfy all the constraints, then a dummy function, such ΣT_{xy} can be used for, say, maximisation.

The simplex method and revised simplex method are the most commonly used methods for LPs. Specialised techniques can be and have been developed to exploit the characteristics of a problem, e.g. problems in which either each inequality has no more than two variables, or all the variables can only take values 0 or 1. For specialised mathematical methods for scheduling, readers are referred to Buchanan [19] for further information.

A particular property of the LP approach is that it will always find the optimal solution if solutions exist. However, when large numbers of disjunctive inequalities are involved, which is often the case in scheduling, the applicability of an LP approach will be limited by the combinatorial explosion problem. Sometimes, it is possible that domain knowledge can be used to reduce the number of disjunctive inequalities in problem formation.

3.2 Depth-first branch-and-bound

Depth-first branch-and-bound (BB) is a well-known and commonly used algorithm for optimisation in OR. It is a general complete search method for finding optimal solutions. It is basically a depth-first search (DFS) plus the use of an evaluation function for pruning off search space. A variation of depth-first BB is an instance of A^* , details of which will not be elaborated upon here — interested readers are referred to Nilsson [20] for further information.

BB searches in the space of states. In scheduling, a state could be an assignment of values to a subset of the variables. For example, the variables could be the starting times in the example scheduling problem in section 1. A neighbourhood function defines the structure of the search space (i.e. which states are directly accessible from which states). For example, the child of a state may assign a time to a variable which has not yet been assigned a value in the parent state. Starting from the root, which makes no assignments, a DSF will visit one child at a time, until all branches have been exhausted. BB remembers the best

value found so far. Before a state is expanded (i.e. looking at its children states as defined by the neighbourhood function), BB will use domain specific knowledge, which is in the form of an evaluation function, to estimate the optimal value under that state. If the estimated value is worse than the best value so far, then the current state will not be expanded. For BB to be sound, the evaluation function must never underestimate (overestimate) the optimal values in a maximisation (minimisation) problem.

BB can be used in combination with the LP approach described in the previous section. In doing so, a state in the search space may represent a subset of conjunctive inequalities. A child state adds an inequality to the set of conjunctive inequalities in the parent state. A leaf state contains a conjunctive set of inequalities, the satisfaction of which would make a complete schedule.

The efficiency of BB is greatly determined by the quality of the estimation of the optimal result under the current branch. The closer this estimation is to the actual values, the more states in the search tree one can potentially avoid searching. Another main factor which determines the number of states that one can avoid searching is the ordering in which the branches are searched. The earlier one can find a tighter bound in a search, the better chance one has in pruning futile branches in the future. It may be worth emphasising that it is only “potential” and “chances” being considered here. The number of states which one can actually avoid searching depends on the structure of the search space, which is decided by the way that the search problem is formulated. Some formulations of a search problem may involve fewer states, smaller branching factors (the average number of children from a state) and/or fewer numbers of common parents than others.

To summarise, BB is mainly applied to problems that require optimisation. Its efficiency is mainly determined by the following factors:

- the way in which the problem is formulated,
- the availability of such evaluation functions and their quality,
- the ordering in which the branches are searched.

Consequently, domain knowledge may help in these three aspects. In general, it too suffers from the combinatorial explosion problem.

3.3 Constraint satisfaction

Constraint satisfaction (CS) is a field of AI research which has demonstrated its usefulness in scheduling. Most constraint satisfaction techniques developed so far are used for satisfiability problems — to find schedules which satisfy

all the constraints. Recent research attempts to develop methods for tackling partial satisfaction problems, problems which require optimisation.

A (finite) constraint satisfaction problem (CSP) comprises a set of variables, a finite domain for each of them, and a set of constraints which restrict the combination of values that the variables can take [10]. The task is to assign to each variable a value from its domain satisfying all the constraints. A large number of methods have been developed for tackling CSPs. Some of these techniques are applicable to problems with infinite domains.

Unlike LP, variables in CSPs are not limited to numerical variables. They can be of numerative type (e.g. John, Mary, Peter, etc). There is no limitation on the kind of constraints that can exist in a CSP (although specialised techniques have been developed for handling certain types of constraints, including numerical constraints, more efficiently). Such flexibility means that constraint satisfaction methods have a wide range of applications.

Constraint satisfaction methods achieve their efficiency by exploring features of CSPs. For example, since the domains are finite and defined in the problem specification, ‘lookahead’ algorithms propagate constraints after assigning a value to each variable in order to reduce the search space and anticipate dead-ends [21]. By using the constraints, intelligent backtracking algorithms analyse the situations in dead-ends so as to backtrack to culprits [22, 23]. By exploring the fact that all the choice points are known before searching starts and are unchanged throughout the search, learning algorithms analyse and record causes of failure so as to avoid wrong choices in the later part of the search [24, 25]. These techniques can also be amalgamated [26]. Specialised algorithms exploit the topology of the constraint graph (a graph which represents the relationship among the variables) of the problem [27, 28].

Given the example scheduling problem in section 1, there are many possible ways to formulate it as a CSP. To allow comparison between constraint satisfaction and LP, the approach examined here is to define a CSP with exactly the same variables as in the LP approach described above, i.e. to use one numerical variable to represent the starting time of each job. The domain of each variable may be initialised to the time in which the corresponding workshop is available. For example, the domain of T_{22} may be initialised to (7..20). However, since “takes 3 hours to process P2, one may shrink the domain of T_{22} to (7.17). Besides, since P2 must be produced between 9:00 and 18:00, one can further shrink the domain of T_{22} to (9..15). This exercise is called non-consistency maintenance [29], which is a problem-reducing exercise [10]. In fact, by further analysing the situation, the domain of T_{22} can be further reduced.

Constraints in this problem can be defined as a function which returns 'true' when the combined assignments are legal, and 'false' otherwise. For example, to ensure that W_2 does not process P_1 and P_2 at the same time, a function can be defined which returns false when:

$$(T_{11} \leq T_{12} < T_{11} + 1) \text{ OR } (T_{12} \leq T_{11} < T_{12} + 3) \quad \dots (5)$$

The first part of function (5) ensures that W_2 does not start processing P_2 while it is processing P_1 (which requires 1 hour), while the second part ensures that it does not process P_1 while processing P_2 (which requires 3 hours).

Based on the above constraint, when [Image] is committed to, say, 12, the values 9 to 12 can be removed from the domain of T_2 . This is another form of problem reduction. Again, the more computation one is prepared to spend to propagate the constraints, the more domains can potentially be reduced in size, and the earlier one can potentially detect dead-ends. A balance has to be established between spending more computation time on constraint propagation and brute force searching.

This example shows that constraint satisfaction techniques are more flexible (e.g. constraints need not be linear inequalities, they can be any function) than LP and, therefore, have a wider domain of application. However, as mentioned earlier, the majority of existing techniques are used for dealing with satisfiability rather than optimisation. Certain constraint satisfaction techniques can be integrated into BB for optimisation. Moreover, the stochastic search techniques discussed in the following subsections have also been applied to constraint satisfaction for optimisation.

Most research in constraint satisfaction so far concentrates on complete search methods. Although some of these methods (e.g. forward checking and the fail-first principle) have been demonstrated to be a lot more efficient than general search methods in scheduling [30, 31], they suffer from the combinatorial explosion problem in general. Domain-specific knowledge may help in several ways. For example, since the number of states in the search space is proportional to $\prod |d_z|$, where Z is the variable in the problem and $|d_z|$ is the domain size of the variable x , domain knowledge could be used to formulate CSPs with a smaller number of variables and domain sizes. Some ways of formulating CSPs result in problems that are easier to solve than others, though problem formulation is more like an art; there has not been much research on methodologies for mastering it. Another common use of domain knowledge is in the ordering of the variables and the ordering of values to try for each variable.

To summarise, constraint satisfaction techniques, which have quite a wide domain of application, are mainly applied to satisfiability problems. A large number of algorithms have been developed, some of which have been demonstrated to be very efficient for certain scheduling problems. However, complete algorithms suffer from the combinatorial explosion problem. Newer and more tractable methods for optimisation in constraint satisfaction research will be described in the following sections.

3.4 Hill climbing

Hill climbing, or local search, is another general search method which has found its way into scheduling. It is a stochastic method, the effectiveness of which relies on heuristics and a certain amount of randomness. It is mainly used for optimisation problems where near-optimal solutions are acceptable, though, as mentioned earlier, satisfiability problems can sometimes be seen as optimisation problems where the number of constraints violated is to be minimised. Near-optimal solutions are often acceptable in real-life applications when the problem is intractable for complete search methods.

Hill climbing requires a neighbourhood function which maps every state to other states in the search space. The definition of this function, which encodes domain specific knowledge, greatly determines the quality of the results that can be obtained.

The basic operation of hill climbing is very simple. Starting with a state in the search space, which is generated either randomly or by some heuristics, pick a neighbour which is 'better' according to the function to be optimised. Then repeat this process until no better state is available from the neighbourhood of the current state. In some hill climbing algorithms, a random better neighbour is picked. In other algorithms, heuristics are used for deciding which better neighbour to move to. For example, in a maximisation (minimisation) problem the neighbour with the highest (lowest) value may be picked; this is called the steepest ascent (descent) heuristic.

Following is one possible hill climbing algorithm for tackling the example scheduling problem given in section 1. Again, to allow comparison, let the task be assigning numerical values to the starting time of the jobs. To generate an initial state, the constraint on the ending time of the workshops' availability may be relaxed. Then a job may be selected for addition to the schedule at a particular time by giving it the earliest starting time subject to the precedence constraints, the availability of the workshops, plus the constraint that no two jobs can be done at the same time by any workshop (refer to constraint (5) in the previous section). To hill climb, resume the constraints on the availability of the workshops. Then, in each iteration, pick a

workshop, the availability constraint of which is violated, and attempt to move any job forward if doing so does not violate more constraints. Failing that, swap the ordering of any two jobs assigned to this workshop if that does not result in violating more constraints. This is a naive algorithm, but it serves to illustrate the hill climbing algorithm. This strategy will also work for applications where the objective is to finish all the jobs as soon as possible.

Local search has received a great deal of attention in recent years. The min-conflict heuristic repair (MCHR) method by Minton et al has been applied to scheduling [33-35] scheduling [36].

Hill climbing is an important technique for tackling problems that are intractable using complete algorithms. It is normally not difficult to invent hill climbing strategies for problems, though finding effective ones may be difficult. One often has to work quite hard to find a strategy for finding solutions that are really near to optimal.

A major limitation of hill climbing is that it may settle in local optima. For example, a search may settle at points B or C in Fig 3, missing the higher point A. A hill climbing search may also wander around in plateaus because, as a local search strategy, there may not be any guidance as to the direction in which it should move. In fact, the example hill climbing algorithm strategy described above may wander around in plateaus by swapping jobs allocated to the same workshop continuously without improving the situation.

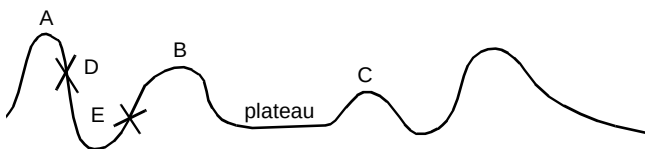


Fig 3 A two-dimensional landscape illustrating the problems of hill climbing.

To summarise, hill climbing is normally used for finding near-optimal solutions in optimisation problems. It sacrifices optimality for speed, and has a wide range of applications. Domain knowledge is heavily relied upon in the design of the neighbourhood function, as this is crucial to the success of a hill climbing algorithm. The existence of local optima and plateaus seriously limit the effectiveness of hill climbing.

3.5 Simulated annealing

Simulated annealing is an extension of hill climbing with the intention to escape local minima. It therefore has the same domain of applications as hill climbing.

The idea of simulated annealing comes from physics [37]. A temperature is initialised to a predetermined starting value, and reduced gradually, according to a cooling schedule, to 0 eventually. Inferior moves are given chances of being accepted. The higher the temperature, the higher the chance of inferior moves being accepted. When the temperature is down to 0, the search behaves in the same way as hill climbing.

The algorithm works as follows. Like hill climbing, the search starts from a state which may be generated randomly or by some heuristic search. In each iteration, a random neighbour is examined. If it is better than the current state, then this new neighbour is made the current state. Otherwise, this neighbour is only accepted under a probability, which is related to the temperature in the above mentioned manner. If it is rejected, then another unexamined neighbour, if one exists, will be examined in the same manner.

This strategy increases the probability of reaching higher points in the search space. For example, in Fig 3, if the starting point is at position E, then standard hill climbing will eventually move to B. But, when the temperature is above a certain level, simulated annealing will allow the search to move to the other side of the valley. However, if by any chance the search moves to any position between A and D, then more energy has to be released in order to move to the right hand side of the valley, which means there is a better chance of reaching position A after the temperature drops to 0.

Like hill climbing, the effectiveness of simulated annealing is significantly determined by the definition of the domain-dependent neighbourhood function. In addition, the cooling schedule sometimes plays an important part. If the temperature is reduced too rapidly, it may not increase the probability of finding better solutions. On the other hand, the slower the temperature is reduced, the longer the time that the algorithm will take to terminate.

3.6 Tabu search

Though developed by the OR community, tabu search resembles the behaviour of hill climbing. It is also used for optimisation applications in which near-optimal solutions are acceptable. Like simulated annealing, the aim of tabu search is also to escape local optima.

¹ Because of the way that problems are represented in GSAT, it is arguably less suitable for general constraint satisfaction problems and scheduling. Debate on this topic is beyond the scope of this paper.

Tabu search should be seen as a class of algorithms. Its power comes from the definition and maintenance of a tabu list. A tabu list disallows certain moves from taking place. For example, it could be the list of the previous k states which have been visited and disallowed from being revisited by the current move. It can also be restrictions on the directions of moves. In the above hill-climbing example, the tabu list may disallow the swapping of jobs for a particular workshop in two consecutive moves. After each move is made, the tabu list will be updated. Different tabu searches may use different strategies to manipulate the tabu list.

Like hill climbing, tabu search starts with a state which may be generated randomly or by heuristic search. At every iteration, the move is always to the neighbour that is both allowed by the tabu list, and has the best value (i.e. it uses the steepest ascent/descent heuristic). This continues until no move is available, the current state is acceptable or resources (such as time) have run out.

By always moving to the best neighbour permitted by the tabu list, tabu search will not get stuck at local optima for the same reason as hill climbing. The tabu list has the responsibility of preventing tabu search from oscillating between a (possibly local) optimum and its neighbour.

The effectiveness of tabu search over hill climbing depends solely on the way that the tabu list is defined and manipulated. Since there is no limit to how that is done, tabu search is a very general strategy.

3.7 Genetic algorithms

Genetic algorithms, which borrow their ideas from evolution and are mainly used for optimisation, are a class of incomplete algorithms which have been demonstrated to be effective in certain scheduling and constraint satisfaction problems [14, 38–40].

To apply GAs to scheduling, a string representation must first be found to represent possible schedules. Binary strings are commonly used. One possible representation for the above example problem is to define a string to represent the starting time of each job. For example, a binary string of 14×5 bits can be used to represent the starting time of the 14 jobs. With 5 bits, this string can represent numbers up to 31. For example:

00011 01000 01010 ... 00111

can be taken to represent the schedule in which $T_{11}=3(00011)$, $T_{12}=8(01000)$, etc. The fitness of a string is taken as the quality of the schedule that it represents.

22 In scheduling, a string generated may not represent legal schedules. One solution to this problem is to impose a

penalty to the fitness of a string to reflect how seriously it has violated the constraints. Another possibility is to 'repair' (e.g. by using hill climbing) any string generated by the algorithm.

Probably a better representation for the above problem is to use the binary string to represent the delay of each job from its earliest possible time. The above string will then be interpreted as $T_{11}=10$, which is 7 (the earliest starting time of P_1) plus 3 (delay of 00011 suggested by the string); $T_{12}=17$ which is 9 (the earliest starting time of P_2) plus 8 (delay of 01000 suggested by the string). This alternative representation encodes the precedence constraint, and therefore, should reduce the space to be searched. The point to make here is that some representations may be better than others, and making the right choice often helps the search tremendously.

One commonly used control strategy of a genetic algorithm is as follows. A random set of strings, which is called a population, is generated. Then, a mating pool is formed by picking randomly weighted strings from the population - the fitter a string, the more chance it is given to enter a mating pool. Pairs of strings are then picked randomly from the mating pool repeatedly to form parents. Parent strings are cut off at a random point and exchanged - this is called crossover. Strings produced this way are subject to occasional mutation, in which random elements of the string may change values. New strings formed this way make the new population. This process is repeated until satisfactory schedules have been generated or resources (such as time) exhausted [12]. An alternative control strategy which is also commonly used is to operate without the mating pool. Strings are picked, weighted randomly, directly from the population, to form parents. New strings generated through crossover and possibly mutation are put back into the population, replacing certain old strings (see Fig 4).

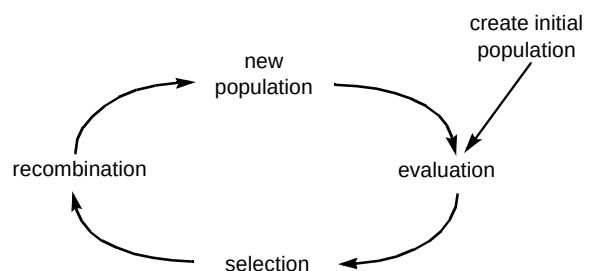


Fig 4 The basic genetic algorithm cycle.

In general, GAs can be expected to stand a better chance of exploring more parts of the search space than hill climbing. However, by their nature, they require a non-trivial number of iterations to find good quality solutions. This means that, in general, a non-trivial amount of time is needed before they can be expected to produce good results.

GAs have to contain the convergence problem, which is akin to the problem of settling in local optima in hill climbing — since those building blocks that appear in relatively fitter strings have a better chance of surviving in the next generation, there is a danger of all the strings having the same building blocks. A balance between exploration (allowing more randomness in the algorithm) and exploitation (e.g. giving more chance to fitter strings to become parents) is crucial to the success of a GA.

A number of parameters have to be chosen in a GA. They include the size of the population and the mating pool, the number of offspring to be generated by each pair of parents, etc. Also, a choice must be made between the GA operators to form the mating pool, crossover, etc. The overall control strategy described above may be changed as well.

To summarise, GAs are used for finding near optimal solutions. In general, they stand a better chance of exploring more parts of the search space (thus finding better solutions) than hill climbing, at the cost of needing more time. Domain knowledge plays an important role in the representation as well as the choice of the many parameters and operations.

3.8 Connectionism

Connectionist methods have been demonstrated to be effective for both satisfiability and optimisation problems. By using a large number of simple processors, they have potential to generate schedules faster than any of the techniques mentioned above. One limitation of connectionist approaches is that building specialised networks for a particular application (in order to gain efficiency) could be costly.

In connectionist methods, a problem is represented by a network. The operations of the nodes in a network and the way that they are connected to each other are the key to the success of this approach. A mechanism is designed to hopefully enable the network to converge to solutions.

Genet is a promising connectionist approach to constraint satisfaction which has been widely published [15, 16, 41, 42]. To apply Genet to scheduling, problems are defined as CSPs, as explained in section 3.3 above.

In Genet, each value of a variable is represented by a label node. Each constraint is represented by a constraint node. Binary constraints are represented by direct inhibitory connections between label nodes. Every non-binary constraint is represented by a constraint node, which is connected to every relevant label node asymmetrically. A set of rules is designed to ensure that the network will settle in certain states. A simple reinforcement learning mechanism is used to escape local optima.

Tests so far on a variety of problems have suggested that Genet has a higher success rate in finding solutions for solvable problems than the best-known hill climbing algorithms published. It can also find solutions more quickly when compared to those algorithms. It is estimated that Genet has the potential of solving quite large problems in seconds. Current research of Genet focuses on optimisation problems.

To summarise, connectionist approaches to scheduling have the potential to solve satisfiability problems very quickly. However, specialised networks for individual applications could be costly. Like constraint-satisfaction techniques, domain knowledge becomes a useful tool in problem formulation.

3.9 Expert systems

Expert systems have a long history of being used for scheduling. One of the best known early works on scheduling is ISIS [17]. In addition, there are probably more published works on this approach than can be enumerated here, most of which are also not worth mentioning because their basic architecture is mainly a rule-based system, and most of their new ideas are application specific. For general introductory references to expert systems, readers may refer, for example, to Jackson [18].

Each expert system is tailor-made for the application for which it is built. Therefore, they can, in principle, be applied to any type of problem that is computable. Rule-based systems have been accepted for scheduling because they are easier to understand. The large majority of the rules used in them come from human experts and therefore can easily be understood by their human users.

The architecture of a rule-based system is very simple. Basically it has a set of rules and a scheduler for conflict resolution — at any time, when more than one rule can fire, the scheduler decides on which rule to fire next. The scheduler in a rule-based system can be as complicated as one wants it to be. It can be a rule-based system itself. The power of an expert system comes from the domain knowledge that is encoded in the rules and the scheduler.

One of the major difficulties in implementing and maintaining a rule-based system is in eliciting rules from human experts. A human expert may be very competent in a scheduling problem, but asking him/her to write down all the relevant rules correctly may be very difficult. Another major difficulty is in defining an appropriate conflict resolution mechanism. Although each individual rule may be sensible, choosing between them the next rule to fire may determine the success of the rule-based system. Moreover, although separating the rules from the inference mechanism may be a good idea, addition and deletion of rules after the

system is built may upset the whole system, and therefore is far from trivial.

Commercial shells for building expert system are abundant. However, if an application requires a more complicated conflict resolution mechanism, or when the rules are difficult to obtain, then an expert system may be quite laborious to build and maintain.

To summarise, expert systems are specially built for their applications and therefore can in, in theory, meet most requirements in scheduling. They obtain their power from domain-specific knowledge. However, building effective systems may be laborious for more complicated applications

3.10 Constraint programming

Some of the above mentioned techniques have been embedded in a number of commercial constraint programming systems. Cras [43] provides an excellent overview of these systems, which provide languages for the problem solvers to express their problems. They also allow the problem solvers to make use of constraint-satisfaction techniques without knowing too much about them, although good knowledge of such techniques will help to improve the efficiency of their software significantly. In addition, these systems provide the problem solver with platforms for building user interfaces and application software without having to start from scratch.

The constraint-programming systems that have made the greatest impact on real life scheduling applications are probably those which take the logic programming approach and those which take the object oriented programming approach. Constraint-logic-programming systems are extended from logic languages. Better known constraint-logic-programming systems are CHIP (and its descendent : ECLⁱPS^e) and Prolog III. By using a logicprogramming syntax, a problem can be expressed in statements which are closer to well-formed formulae in predicate calculus; also backtracking is built into these systems. Numerical constraints and optimisation are handled by linear programming and branch-and-bound techniques in these systems. For symbolic variables and constraints, lookahead algorithms (which are complete search algorithms) in constraint satisfaction are used. Although new and specialised techniques have been developed to help prune the search space, these systems suffer from the same limitation as both linear programming and complete search techniques in constraintsatisfaction. Interested readers should refer to Lever, Wallace and Richards [44] for recent developments in ECLⁱPS^e.

Ilog solver takes the object oriented programming style. It uses C++ syntax, which allows it to fit into more existing applications than logic programming systems, e.g. by allowing the programs to share common objects and libraries. It uses similar constraint-satisfaction techniques to handle symbolic constraints, but allows the problem solver to define its own control strategies. This means the problem solver can choose to use stochastic search methods (such as hill climbing, simulated annealing, etc) if they are considered to be appropriate (e.g. for containing the combinatorial explosion problem). However, the user has to be an expert in scheduling techniques in order to take advantage of such flexibility.

4. Contrasting scheduling techniques

To decide on which technique to use for a given problem, one needs in-depth knowledge about the problem as well as the techniques. In section 2, a list of important questions to ask when facing a scheduling problem was suggested. The previous section summarised a number of techniques and explained when they are useful. This section will contrast these techniques (based on the discussions in section 3) with reference to the question list in section 2.

4.1 Satisfiability or optimisation

On the question of optimality, LP, BB, hill climbing, simulated annealing and tabu search are mainly used for optimisation. Most constraint satisfaction research and connectionist approaches to scheduling have focused on satisfiability problems. Expert systems are built to serve any particular need. When optimisation is required, LP can only be applied to problems where constraints and objective functions are linear.

Scheduling problems normally involve disjunctive constraints. LP and BB guarantee to find optimal solutions if time is not a problem. Hill climbing, simulated annealing and tabu search are designed to sacrifice completeness for speed. Therefore, solutions returned by them may be sub-optimal.

In scheduling, it is often the case that some constraints must be satisfied (e.g. the maximum speed of a printing machine and the capacity of an oven are fixed), while the satisfaction of other constraints is preferred but not necessary (e.g. the work-force in certain jobs, which can be increased temporarily by overtime and employing temporary workers). The former are called hard constraints and the latter soft constraints. One way to handle soft constraints is to treat them like hard constraints. Then the whole problem is treated as a satisfiability problem. When it is not possible to find a schedule which satisfies all the constraints, soft constraints can be selected and relaxed (it would help if the algorithm used is able to identify critical

constraints). Another way to handle soft constraints is to assign costs to the violation of each constraint. Then the problem is treated as an optimisation problem in which the cost of violating the constraints is to be minimised.

4.2 *On tractability and response time*

On the aspect of computation time, LP, BB and complete search algorithms in constraint satisfaction are limited by the combinatorial explosion problem. They need exponential time as the size of the problem grows (in the case of LP, the size of the problem is measured by the number of disjunction constraint sets and their sizes). However, in some situations, ample time is allowed for scheduling, and optimal solutions are preferred (especially in long-term planning which involves expensive resources). In other cases, missing solutions could be extremely costly (for example in medical diagnosis). In those cases, complete algorithms should at least be tried.

In applications such as emergency handling when no preplanned procedures are applicable, generating suboptimal solutions very quickly may be more useful than generating optimal solutions in a very long period of time. In such applications, interactive scheduling is needed. Connectionist approaches stand the best chance of responding quickly, followed by hill climbing. Simulated annealing and tabu search would normally require more time than hill climbing, which is a price to pay for finding better solutions.

4.3 *Problem specification and formulation issues*

Before choosing among the various techniques, the scheduling problem must be specified and formulated. In summarising the techniques in section 3, it was explained how the typical scheduling problem in section 2 could be formulated for those techniques to be applied. In this section, a more general analysis is given.

LP may handle problems which can be specified by a set of linear inequalities together with an objective function. BB can be applied to general optimisation problems where the means to evaluate the quality of a partial solution is available; this seldom poses any problem, but the efficiency of a search is partly determined by the accuracy of such evaluations. Most conventional constraint-satisfaction techniques are applied to finite domain problems, though some of them can be applied to problems with infinite domains. Hill climbing, simulated annealing and tabu search can be applied to a wide range of problems; their effectiveness depends on the neighbourhood functions that the problem solver manages to find. The effectiveness of a GA depends heavily on how candidate solutions are represented and how the evaluation function is defined,

which requires expertise from the problem solver. Connectionist methods have been shown to be applicable to problems which can be formulated as finite constraint-satisfaction problems with or without optimisation requirements.

The nature of the problem and the domain knowledge available often limits the way that the problems can be formulated. Now that these techniques have been summarised, some more questions can be added to the list in section 2.

- Are the variables all numerical?
- Are all constraints linear inequalities, or are they complicated functions?
- Can a good evaluation function be found for BB to apply?
- Can a neighbourhood function be found which allows a search for solutions to be made effectively?
- Can a good representation and evaluation function be found for GAs to be applied?

4.4 *Algorithm design/selection and implementation issues*

The choice of techniques for a particular problem is also affected by the difficulty of designing (or selecting) algorithms and implementation.

Algorithms in LP, BB and constraint satisfaction are mostly well defined in the literature. Therefore, the implementation of them does not require a lot of ingenuity by the problem solver, though this does not mean that it is always an easy task. The complication is, however, that even after constraint satisfaction has been chosen as the technique to use, a choice still has to be made between the large number of algorithms developed in this field; this requires a great deal of knowledge.

Although the basic hill climbing algorithm is well defined, its effectiveness depends on the neighbourhood function and heuristics (for determining hill climbing directions). The problem solver is expected to provide them. Finding a neighbourhood function and heuristics for a problem is normally not difficult, but finding those which allow the space to be searched effectively is often hard. In addition to the neighbourhood function and heuristics, simulated annealing requires the definition of a cooling schedule, which is normally not very difficult to find. The difficulty in the implementation of hill climbing and simulated annealing is mostly determined by the complexity of the neighbourhood function and heuristics used. The difficulty in the implementation of tabu search algorithm is

mostly determined by the same, plus the complexity of the tabu list and its manipulation mechanism.

Expert systems can be built from shells relatively easily, but finding effective rules and conflict-resolution mechanisms is far from easy. Constraint-programming systems enable the problem solver to use LP, BB and basic constraint-satisfaction techniques to scheduling without having to learn and implement them.

4.5 Remarks

Choosing among the above-mentioned scheduling techniques is far from trivial and it is not being suggested here that a choice can be made mechanically. Every problem is unique. Some of them allow the opportunity for applying specialised algorithms. Moreover, the problem solver's knowledge about the techniques and the application domain greatly influence the choice of the techniques. Therefore, this paper has provided only a set of general guidelines. In practice, every problem should be handled individually in choosing between the scheduling techniques.

Different techniques would use different types of domain knowledge. Therefore, it is important to remember that the types of domain knowledge available hereoften plays the most important part in the choice of techniques.

Choosing a system (which may embed various techniques) for scheduling is also a complicated task. For example, whether the system allows the problem to be expressed easily, and the usual considerations such as the speed and reliability of the system, documentation, interface with existing software, support, etc, are all important. Providing guidance for choosing between scheduling systems goes beyond the scope of this paper.

5. Conclusions

This paper has enumerated and summarised some of the better-known scheduling techniques. The focus has been on when they are applicable, and how to choose between them; a list of important questions to ask when facing a problem has also been suggested (see Table 2 for summary).

Problem solving is a complex activity. Domain knowledge often plays the most important part in it, and therefore, each problem should be handled individually. Generalisation of the situations in which each technique is applicable is next to impossible. Therefore, analysis in this paper may only serve as guidelines. Having said that, knowledge of the techniques is essential for choosing between them. Providing a basic knowledge of scheduling techniques has been one of the objectives of this paper.

Table 2 Considerations in choosing between major scheduling techniques.

	General considerations (see section 2)	Major technique-specific considerations
Linear programming	- Used for optimisation with linear functions - Intractable	Problem must be specified by a (normally conjunctive set of equalities
Branch-and-bound	- Used for optimisation - Intractable	- Requires heuristic for pruning - Ordering of branches is important
Constraint satisfaction	- Most existing algorithms used for finding single or all solutions satisfying constraints - Both complete and incomplete algorithms available	Large number of algorithms available particularly useful when problem involves non-trivial amount of constraints
Hill climbing	Useful for both constraint satisfaction and optimisation when near-optimal solutions are acceptable	Requires a neighbourhood function which is crucial to its effectiveness
Simulated annealing	- Flexible in computation time, which makes them widely useful - Hill climbing could be trapped in local optima	- Neighbourhood function is crucial to its effectiveness - Cooling schedule could be important
Tabu search	Simulated annealing and tabu search attempt to escape from local optima	Effectiveness mainly depends on strategy on tabu-list manipulation
Genetic algorithms	- Useful for satisfiability problems or for finding near-optimal solutions - Good potential for parallel implementation which may suit real time applications	- Set up and network updating mechanism are crucial to its effectiveness - Specialised network may be expensive to build
Expert systems	- Wide range of applicability; can be tailor-made to meet the requirements (including time and optimality requirements) - Power comes from domain-specific knowledge	- Expert knowledge elicitation is important and may be difficult - Conflict resolution may be non-trivial

References

- 1 Freuder E C: 'A sufficient condition for backtrack-free search', *J ACM*, **29**, No 1, pp 24—32 (1982).
- 2 Lawler E W and Wood D E: 'Branch-and-bound methods: a survey', *Operations Research*, **14**, pp 699—719 (1966).
- 3 Hall P A V: 'Branch-and-bound and beyond', *Proc International Joint Conference on AI*, pp 641—650 (1971).
- 4 Reingold E M, Nievergelt J and Deo N: 'Combinatorial algorithms: theory and practice', Englewood Cliffs, NJ, Prentice Hall (1977).
- 5 Glover F: 'Tabu search: part I', *Operations Research Society of America (ORSA), Journal on Computing*, **1**, pp 109—206 (1989).
- 6 Glover F: 'Tabu search: part II', *Operations Research Society of America (ORSA), Journal on Computing*, **2**, pp 4—32 (1990).
- 7 Aarts E and Korst J: 'Simulated annealing and Boltzmann machines', John Wiley & Sons (1989).
- 8 Davis L (Ed): 'Genetic algorithms and simulated annealing', *Research notes in AI*, Pitman/Morgan Kaufmann (1987).
- 9 Otten R H J M and Van Ginneken L P P P: 'The annealing algorithm', Kluwer Academic Publishers (1989).
- 10 Tsang E P K: 'Foundations of constraint satisfaction', Academic Press (1993).
- 11 Holland J H: 'Adaptation in natural and artificial systems', University of Michigan Press, MI (1975).
- 12 Goldberg D E: 'Genetic algorithms in search, optimization, and machine learning', Addison-Wesley Pub Co Inc (1989).
- 13 Davis L (Ed): 'Handbook of genetic algorithms', Van Nostrand Reinhold (1991).
- 14 Tsang E P K and Warwick T: 'Applying genetic algorithms to constraint satisfaction problems', *Proceedings of the European Conference on AI*, pp 649—654 (1990).
- 15 Wang C J and Tsang E P K: 'Solving constraint satisfaction problems using neural networks', *Proceedings of IEE Second International Conference on Artificial Neural Networks*, pp 295—299 (1991).
- 16 Davenport A, Tsang E P K, Wang C J and Zhu K: 'GENET: a connectionist architecture for solving constraint satisfaction problems by iterative improvement', *Proc 12th National Conference for Artificial Intelligence (AAAI)*, pp 325—330 (1994).
- 17 Fox M: 'Constraint-directed search: a case study of job-shop scheduling', Morgan Kaufmann (1987).
- 18 Jackson P: 'Introduction to expert systems', 2nd Edition, Addison Wesley (1990).
- 19 Buchanan I: 'Mathematical programming and resource management', *BT Technol J*, **13** No 1, pp 29—37 (January 1995).
- 20 Nilsson N J: 'Principles of artificial intelligence', Tioga Publishing Co, Palo Alto, California (1980).
- 21 Haralick RM and Elliott G L: 'Increasing tree search efficiency for constraint satisfaction problems', *Artificial Intelligence*, **14** pp 263—313 (1980).
- 22 Gaschnig J: 'A problem similarity approach to devising heuristics: first results', *Proc 6th International Joint Conference on AI*, pp 301—307 (1979).
- 23 Ginsberg M: 'Dynamic backtracking', *Journal of Artificial Intelligence Research*, **1** pp 25—46 (1993).
- 24 Dechter R: 'Learning while searching in constraint-satisfaction problems', *Proc National Conference for Artificial Intelligence (AAAI)*, pp 178—183 (1986).
- 25 Frost D and Dechter R: 'Dead-end driven learning', *Proc 12th National Conference for Artificial Intelligence (AAAI)*, pp 294—300 (1994).
- 26 Prosser P: 'Hybrid algorithms for the constraint satisfaction problem', *Computational Intelligence*, **9**, No 3, pp 268—299 (1993).
- 27 Dechter R and Pearl J: 'Network-based heuristics for constraintsatisfaction problems', *Artificial Intelligence*, **34**, pp 1—38 (1988).
- 28 Dechter R and Pearl J: 'Tree clustering for constraint networks', *Artificial Intelligence*, **38**, No 3, pp 353—366 (1989).
- 29 Mackworth A K: 'Consistency in networks or relations', *Artificial Intelligence*, **8**, No 1, pp 99—118 (1977).
- 30 Dincbas M, Simonis H and Van Hentenryck P: 'Solving large combinatorial problems in logic programming', *Journal of Logic Programming* (1987).
- 31 Dincbas M, Simonis H and Van Hentenryck P: 'Solving car sequencing problem in constraint logic programming', *Proceedings of the European Conference on AI*, pp 290—295 (1988).
- 32 Minton S, Johnston M D, Philips A B and Laird P: 'Solving largescale constraint-satisfaction and scheduling problems using a heuristic repair method', *Proc National Conference for Artificial Intelligence (AAAI)*, pp 17—24 (1990).
- 33 Selman B, Levesque H and Mitchell D: 'A new method for solving hard satisfiability problems', *Proc National Conference on Artificial Intelligence (AAAI)*, pp 440—446 (1992).
- 34 Selman B and Kautz H: 'Domain-independent extensions to GSAT: solving large structured satisfiability problems', *Proc 13th International Joint Conference on AI*, pp 290—295 (1993).
- 35 Selman B, Jautz H A and Cohen B: 'Noise strategies for improving local search', *Proc 12th National Conference for Artificial Intelligence (AAAI)*, pp 337—343 (1994).
- 36 Yugami N, Ohta Y and Hara H: 'Improving repair-based constraint satisfaction methods by value propagation', *Proc 12th National Conference for Artificial Intelligence (AAAI)*, pp 344—349 (1994).
- 37 Kirkpatrick S, Galatt C D and Becchi M P: 'Optimization by simulated annealing', *Science*, **220** pp 671—680 (1983).
- 38 Cleveland G A and Smith S F: 'Using genetic algorithms to schedule flow shop releases', in Schaffer J D (Ed): 'Proceedings of the 3rd International Conference on Genetic Algorithms', Morgan Kaufmann, pp 160—169 (1989).
- 39 Whitley D, Starkweather T and Fuquay D: 'Scheduling problems and travelling salesmen: the genetic edge recombination operator', in Schaffer J D (Ed), 'Proceedings of the 3rd International Conference on Genetic Algorithms', Morgan Kaufmann, pp 133—140 (1989).
- 40 Warwick T and Tsang E P K: 'Using a genetic algorithm to tackle the processors configuration problem', *Symposium on Applied Computing* (1994).

SCHEDULING TECHNIQUES

- 41 Wang C J and Tsang E P K: 'A cascable VLSI design for GENET', International Workshop on VLSI for Neural Networks and Artificial Intelligence', Oxford (1992).
- 42 Tsang E P K and Wang C J: 'A generic neural network approach for constraint satisfaction problems', in Taylor J G (Ed): "Neural Network Applications", pp 12—22, Springer-Verlag (1992).
- 43 Cras J Y: 'A review of industrial constraint solving tools', AI Intelligence (1993).
- 44 Lever J, Wallace M and Richards B: 'Constraint logic programming for scheduling and planning', BT Technol J, 13 No 1, pp 75—82 (January 1995).



Edward Tsang is a Senior Lecturer at the University of Essex.

He is an active researcher in constraint satisfaction. He is the author of the book Foundations of Constraint Satisfaction, and the principal investigator of two EPSRC funded projects.